



华中科技大学

操作系统原理课程实验报告

姓 名: Losyi
学 院: 计算机科学与技术学院
专 业: 计算机科学与技术
班 级: CS23
学 号: U2023xxxxx
指导教师: 邵志远

分数	
教师签名	

2025 年 12 月 20 日

目 录

实验一 系统调用、异常和外部中断	1
1.1 实验目的	1
1.2 实验内容	1
1.3 实验调试及心得	2
1.3.1 实验收获	2
1.3.2 实验过程（AI 交互和调试过程）	3
1.3.3 实验建议	13
实验二 内存管理	14
2.1 实验目的	14
2.2 实验内容	14
2.3 实验调试及心得	15
2.3.1 实验收获	15
2.3.2 实验过程（AI 交互和调试过程）	15
2.3.3 实验建议	19
实验三 进程管理	21
3.1 实验目的	21
3.2 实验内容	21
3.3 实验调试及心得	22
3.3.1 实验收获	22
3.3.2 实验过程（AI 交互和调试过程）	22
3.3.3 实验建议	32
实验四 文件系统	33
4.1 实验目的	33
4.2 实验内容	33
4.3 实验调试及心得	34
4.3.1 实验收获	34
4.3.2 实验过程（AI 交互和调试过程）	34
4.3.3 实验建议	39
附录	41

实验一 系统调用、异常和外部中断

1.1 实验目的

本实验围绕 RISC-V 架构下的代理内核（PKE），通过系统调用、异常和外部中断等典型机制，引导学生从底层执行流程的角度理解操作系统内核与硬件之间的协作关系。实验的设计重点不在于实现功能的复杂性，而在于通过动手补全关键内核逻辑，使学生建立对特权级切换、Trap 处理以及执行上下文保存与恢复的整体认识。作为 PKE 实验的开始，该实验引导学生了解 RISC-V 程序的编译和链接，代理内核的构造和启动，ELF 文件，HIHF 接口等基础知识。

在基础实验部分，通过实现系统调用，学生可以理解用户态程序如何在受限环境下请求内核服务，以及参数与返回值如何依托寄存器和 Trapframe 在用户态与内核态之间传递。异常处理实验进一步说明了当用户程序执行越权或非法指令时，硬件与内核如何共同工作以阻止错误扩散，从而体现操作系统在安全隔离中的作用。外部中断实验则通过时钟中断的处理流程，使学生认识到内核不仅需要响应同步异常，还必须正确处理异步事件，并在中断完成后恢复程序的正常执行。

在此基础上，挑战实验 lab1_challenge1 “打印用户程序调用栈”进一步提升了实验的深度。该挑战要求学生结合函数调用约定、栈帧结构以及 ELF 符号表，对运行中的用户程序进行栈回溯和符号解析，从而将“程序执行状态”与“静态二进制结构”联系起来。通过这一实验，学生能够更深入地理解函数调用过程、栈布局以及调试信息在操作系统中的实际价值，强化对程序运行时行为的整体把握，为后续调试与内核扩展能力的培养奠定基础。

1.2 实验内容

本实验围绕 PKE 实验一展开，实验内容主要包括系统调用处理、异常处理、外部中断处理以及扩展实验：打印用户程序调用栈四个部分。实验采用本地开发环境，通过 Docker 容器和 VSCode 完成代码编写、编译、运行与调试，并通过 Git 对实验过程进行版本管理和记录。

实验过程使用 AI 辅助开发，各项实验均一次通过评测，无需调试修改，具体实验过程将在 1.3 介绍。

第1关 lab1_challenge1: (挑战实验) 打印用户程序函数调用栈

学员代码 代码修改行数 199 kernel/strap.c 复制代码

```
4
5 #include "riscv.h"
6 #include "process.h"
7 #include "strap.h"
8 #include "syscall.h"
9
10 #include "spike_interface/spike_utils.h"
11
12 //
13 // handling the syscalls. will call do_syscall() defined in kernel/syscall.c
14 //
15 static void handle_syscall(trapframe *tf)
16 {
17     // 修正返回地址, 跳过 ecall 指令
18     tf->epc += 4;
19 }
```

评测次数	详细信息	评测耗时(S)	内存消耗(MB)	评测时间	操作
1	评测通过	2.31	156.82	2025-12-06 00:13	查看代码

图 1-1 实验一头歌评测通过图

1.3 实验调试及心得

1.3.1 实验收获

1. 学会借助 AI 辅助完成实验，实现“0 代码”理解式开发

本实验全程使用 **Gemini 3.0 pro** 作为 agent 辅助开发，实现“0 代码”完成实验，通过 prompt，让 AI 直接指出需要修改的代码位置和修改内容，同时给出实验原理和修改思路的解释，在此基础上由自己判断和验证修改结果。该过程在一定程度上实现了“0 代码”层面的理解式开发，即重点放在对执行流程和设计意图的理解上，而非编写代码，这显著提高了学习效率。

2. 对操作系统基础知识的理解更加具体和系统

通过实验一，我对实验介绍文档 3.1 部分中涉及的基础知识有了更加直观的认识，包括 RISC-V 程序的编译与链接过程、ELF 文件的结构、代理内核的启动流程以及异常和中断的处理机制。相比仅停留在理论层面的学习，本实验通过代码执行路径和运行结果，将“系统调用”“异常”“中断”等抽象概念具体化，使我能够从硬件触发、内核处理到程序恢复执行的完整流程中理解其工作原理。

3. 掌握了基于 Docker 和 VS Code 的实验开发方式

在实验过程中，我完成了 Docker 环境的搭建与使用，并结合 VS Code 进行代码浏览、修改和调试。这一过程使我熟悉了基于容器的开发模式，避免了环

境差异带来的问题，也提高了实验的可复现性。同时，通过在容器中使用命令行工具进行编译、运行和调试，我对操作系统实验中常见的开发流程有了更深入的掌握。

1.3.2 实验过程（AI 交互和调试过程）

（1）lab1_1 系统调用

图 1-3 展示了与 Gemini 的交互记录，通过与 AI 交互，明确了 `ecall` 指令触发系统调用的流程，以及 `Trapframe` 在参数传递和返回值保存中的作用。在 AI 的引导下，定位到需要修改的函数（`kernel/strap.c` 的 `handle_syscall` 函数），并理解了为何必须在处理完系统调用后调整 `EPC`。

```
用户：我正在进行计算机操作系统PXE实验课程，请你阅读实验文档，指导我完成lab1_1，指出需要修改的代码和实验过程。
Gemini：
好的，按照 lab1_2 的网络格式，以下是 lab1_1：系统调用的详细指导和原理分析。

1. 实验原理与分析过程（为什么要这样做？）
要完成这个实验，需要理解操作系统如何处理用户程序的系统调用（System Call）请求，以及 Trapframe（中断帧）在用户态和内核态之间传递数据的角色。

A. 触发原因
应用程序 user/app_helloworld.c 调用了 printf 函数，最终在 user/user_1ib.c 的 do_user_eca11 函数中执行了汇编指令 eca11。
• 指令作用：eca11 指令用于触发中断，通知操作系统内核用户程序请求服务（例如打印字符或退出程序）。
• 参数传递：在执行 eca11 之前，函数参数已经按照 RISC-V 调用约定放入了通用寄存器 a0 到 a7 中。

B. 异常流向（Trap Flow）
1. 陷入内核：eca11 执行后，CPU 从用户模式（U-mode）切换到特权模式（这里是 S-mode，因为 PXE 将前阶段代理给了 S 模式），并跳转到 kernel/strap_vector.S 中的入口 smode_trap_vector。
2. 保存现场：smode_trap_vector 会将当前的通用寄存器（包括存放参数的 a0 - a7）保存到当前进程的 Trapframe 结构体中。Trapframe 是内存中的一块区域，用于保存用户程序的上下文。
3. 分发处理：接着跳转到 C 语言处理函数 smode_trap_handler（位于 kernel/strap.c）。该函数读取 scause 寄存器，发现原因是 CAUSE_USER_ECALL，于是调用 handle_syscall 函数。

C. 处理逻辑（Handling Logic）
目前的 handle_syscall 函数中只有一个 panic 语句，导致程序直接终止。你需要完成真正的系统调用处理逻辑：
1. PC 指针修正：系统调用属于同步异常，处理完后返回用户态时，程序计数器（PC/EPC）必须指向 eca11 的下一条指令，否则会无限重复执行 eca11。因此代码中已有 tf->epc += 4;（RISC-V 指令长度为 4 字节）。
2. 调用内核功能：你需要调用 kernel/syscall.c 中已定义好的 do_syscall 函数来执行具体操作。
3. 数据传递：
   • 参数：do_syscall 需要参数，这些参数都在寄存器里，现在保存在 tf->regs 结构中，你需要将 tf->regs.a0 到 tf->regs.a7 传给 do_syscall。
   • 返回值：do_syscall 会返回执行结果（例如成功或失败代码），你需要将这个返回值写回 tf->regs.a0。这样当内核恢复现场返回用户态时，用户程序的 a0 寄存器里就会有正确的返回值。

2. 需要修改的代码位置
• 文件路径：kernel/strap.c
• 函数名称：handle_syscall

3. 代码修改逻辑
定位到 handle_syscall 函数，进行如下修改：

修改前：
static void handle_syscall(trapframe *tf) {
    tf->epc += 4; // 已有的指令地址修正

    // TODO (lab1_1): remove the panic call below...
    panic("call do_syscall to accomplish the syscall and lab1_1 here.\n");
}

修改后：
static void handle_syscall(trapframe *tf) {
    // 修正返回地址：跳回 eca11 指令
    tf->epc += 4;

    // 1. 调用 do_syscall。将保存的 Trapframe 中的用户态寄存器值作为参数传递
    // do_syscall 在 kernel/syscall.c 中定义
    long ret = do_syscall(tf->regs.a0, tf->regs.a1, tf->regs.a2, tf->regs.a3,
                        tf->regs.a4, tf->regs.a5, tf->regs.a6, tf->regs.a7);

    // 2. 将系统调用的返回值存回 Trapframe 的 a0 寄存器
    // 这样当从 Trap 返回时，用户程序能从 a0 获取返回值
    tf->regs.a0 = ret;
}
```

图 1-2 lab1_1 交互记录

根据 AI 给出的答案，定位到 `handle_syscall` 函数，进行如下修改：
修改前：

```
static void handle_syscall (trapframe *tf) {
    tf->epc += 4; // 已有的指令地址修正
    // TODO (lab1_1): remove the panic call below...
    panic
    ( "call do_syscall to accomplish the syscall and lab1_1 here.\n" );
}
```

修改后：移除 panic，调用 do_syscall 并处理参数与返回值。

```
static void handle_syscall (trapframe *tf) {
    // 修正返回地址，跳过 ecall 指令
    tf->epc += 4;
    // 1. 调用 do_syscall，将保存在 Trapframe 中的用户态寄存器值作为参数传递
    // do_syscall 在 kernel/syscall.c 中定义
    long ret = do_syscall
    (tf->regs.a0, tf->regs.a1, tf->regs.a2, tf->regs.a3,
    tf->regs.a4, tf->regs.a5, tf->regs.a6, tf->regs.a7);
    // 2. 将系统调用的返回值写回 Trapframe 的 a0 寄存器
    // 这样当从 Trap 返回时，用户程序能从 a0 获取返回值
    tf->regs.a0 = ret;
}
```

修改完成后，编译项目并运行仿真，可见程序正确输出了 "Hello world!" 并正常退出，而不是报错，如图 1-4 所示。

```
root@015b67a0fbff:/app/riscv-pke# spike ./obj/riscv-pke ./obj/app_helloworld
In m_start, hartid:0
HTIF is available!
(Emulated) memory size: 2048 MB
Enter supervisor mode...
Application: ./obj/app_helloworld
Application program entry point (virtual address): 0x0000000081000000
Switch to user mode...
Hello world!
User exit with code:0.
System is shutting down with exit code 0.
```

图 1-3 lab1_1 运行结果

(2) lab1_2 异常处理

图 1-5 展示了与 Gemini 的交互记录，AI 帮助分析了非法指令异常未被委托给 S 态的原因，使我理解该异常必须在 M 态处理中捕获。在实验中，通过对比 panic 输出与修改后的运行结果，我加深了对异常分发机制的认识。

用户：继续指导我完成lab1_2除了说明答案（修改代码位置）外，你还应该给出分析过程，让我明白实验原理（即为什么这么做）

Gemini:

好的，我们继续进行 lab1_2: 异常处理。

在这个实验中，你需要处理由用户程序执行非法指令 (illegal instruction) 的情况。

1. 实验原理与分析过程（为什么要这样做？）

要完成这个实验，必须理解 RISC-V 的特权级 (Privilege Mode) 和异常委托 (Delegation) 机制。

A. 触发原因

给定的用户程序 user/app_illegal_instruction.c 试图执行汇编指令 core_sscratch, 0。

- 问题所在：csrw 是与控制状态寄存器 (CSR) 的指令，而 sscratch 是特权寄存器。
- 触发原因：用户程序运行在 U 模式 (User Mode)，而该指令需要更高的权限 (如 S 模式或 M 模式)。因此，CPU 会触发一个非法指令异常 (Illegal Instruction Exception)。

B. 异常流向 (Trap Flow)

当异常发生时，硬件会将控制权交给内核。关键在于交给谁处理？

- 委托机制设置：RISC-V 默认将所有异常交给最高权限的 M 模式 (Machine Mode)。虽然 PxE 内核在启动时调用了 delegate_trap() 函数将部分异常 (如断点、断点等) 委托给了 S 模式处理。
- 发现委托链：检查 kernel/machine/interrupt.c 中的 delegate_trap() 代码，你会发现 CAUSE_ILLEGAL_INSTRUCTION 异常委托包含在委托列表 exceptions 中。
- 结论：这意味着该异常必须由 M 模式的中断处理程序来捕获和处理，而不是 S 模式。

C. 代码定位

- M 模式的入口是 kernel/machine/exception.S 中的 mtrapvec。
- mtrapvec 会调用 C 语言编写的处理函数 handle_mtrap，位于 kernel/machine/mtrap.c。
- 在 handle_mtrap 函数中，有一个 switch-case 结构根据 cause (异常原因) 分发任务。当前代码在 case CAUSE_ILLEGAL_INSTRUCTION 分支中直接调用了 panic，导致系统挂起。

2. 需要修改的代码位置

你需要修改的是 M 模式下的异常处理分发函数。

- 文件路径：kernel/machine/mtrap.c
- 函数名称：handle_mtrap

3. 代码修改逻辑

找到 handle_mtrap 函数中的 switch 语句，定位到 case CAUSE_ILLEGAL_INSTRUCTION 分支。

修改前：

```
case CAUSE_ILLEGAL_INSTRUCTION:
    // TODO (lab1_2): call handle_illegal_instruction ...
    panic("call handle_illegal_instruction to accomplish illegal instruction interception for lab1_2.\n");
    break;
```

修改后：

将 panic 替换为对 handle_illegal_instruction() 的调用。这个函数（虽然文档未展示其具体实现细节，但已在内核中定义）负责安全地终止违规的进程或打印错误信息。

```
case CAUSE_ILLEGAL_INSTRUCTION:
    // 调用处理非法指令的函数
    handle_illegal_instruction();
    break;
```

图 1-4 lab1_2 交互记录

根据 AI 的回复，我们知道需要修改的是 M 模式下的异常处理分发函数，即 kernel/machine/mtrap.c 路径下的的 handle_mtrap 函数。

修改前：

```
case CAUSE_ILLEGAL_INSTRUCTION:
    // TODO (lab1_2): call handle_illegal_instruction ...
    panic
( "call handle_illegal_instruction to accomplish illegal instr
uction interception for lab1_2.\n" );
    break;
```

修改后：将 panic 替换为对 handle_illegal_instruction () 的调用。这个函数（虽然文档未展示其具体实现细节，但已在内核中定义）负责安全地终止违规的进程或打印错误信息。

```
case CAUSE_ILLEGAL_INSTRUCTION:
    // 调用处理非法指令的函数
    handle_illegal_instruction ();
    break;
```

修改完成后，进行编译和运行，可以看到系统捕获了异常，打印 "Illegal instruction!" 并以非 0 状态码退出，而不是打印 Panic 信息，如图 1-6 所示。

```
root@015b67a0fbff:/app/riscv-pke# spike ./obj/riscv-pke ./obj/app_illegal_instruction
In m_start, hartid:0
HTIF is available!
(Emulated) memory size: 2048 MB
Enter supervisor mode...
Application: ./obj/app_illegal_instruction
Application program entry point (virtual address): 0x0000000081000000
Switch to user mode...
Going to hack the system by running privilege instructions.
```

图 1-5 lab1_2 运行结果

总之，lab1_2 的核心是让我们理解并非所有的异常都会进入操作系统（S 模式）。有些底层的、致命的或者特定的异常（取决于 OS 设计）会直接由 M 模式（固件/硬件抽象层）拦截。在这里，PKE 选择在 M 模式下直接拦截非法指令。

（3）lab1_3 异常处理

图 1-7 展示了与 Gemini 的交互记录（记录过长，截图未全部展示回复内容），在该实验中，AI 的分析重点放在时钟中断的转发流程以及中断标志清除的重要性上。我通过实际运行验证了如果未清除中断标志会导致系统反复陷入中断，从而印证了 AI 给出的分析结论。

用户：继续lab1_3

Gemini：

好的，按照之前的格式，以下是 lab1_3：中断处理的详细指导和原理分析。

1. 实验原理与分析过程（为什么要这样做？）

本实验的目标是修复内核对时钟中断（Timer Interrupt）的处理逻辑，使得操作系统能够正确响应时间片轮转信号，而不是在第一个中断到来时就崩溃。

A. 触发原因

应用程序 user/app_long_loop.c 执行一个长循环。在此期间，RISC-V 机器的定时器硬件会周期性地产生中断。

- **M 态介入**：时钟中断首先被 M 模式捕获（因为 delegate_traps 没有代理时钟中断）。M 模式的处理函数 handle_timer 会重置下一个时钟点，并通过设置 sip 寄存器的 SIP_SSIP 位，向 S 模式发送一个“软中断”信号，模拟 S 模式的时钟中断。
- **S 态接收**：当 M 模式处理完毕返回后，S 模式检测到 sip 寄存器中有挂起的中断（即刚才设置的软中断），于是再次陷入异常，进入 smode_trap_handler。
- **分发处理**：smode_trap_handler 发现 scause 为 CAUSE_MTIMER_S_TRAP（即 M 态转发过来的时钟中断），从而调用 handle_mtimer_trap。

B. 问题所在

当前的 handle_mtimer_trap 函数不完整，只打印了一句 Panic 信息就停止了。

如果只移除 panic 而不做其他处理，会发生死循环中断：

1. S 模式处理完中断返回。
2. 由于 sip 寄存器里的 SIP_SSIP 位（中断挂起标志）仍然是 1（未被清除）。
3. CPU 认为还有一个中断等待处理，于是立即再次进入 smode_trap_handler。
4. 程序永远无法回到用户态继续执行 app_long_loop。

C. 处理逻辑（Handling Logic）

你需要完成 handle_mtimer_trap 的后续动作：

1. **更新计数器**：增加全局变量 g_ticks 的值，用于记录系统经过了多少个时间片。这是操作系统进行进程调度和计时的基础。
2. **清除中断标志**：必须手动清除 sip 寄存器中的 SIP_SSIP 位。这就相当于告诉 CPU：“这个中断我已经处理过了，请把红灯熄灭”。这样返回用户态后，CPU 才会继续执行应用程序，直到下一次 M 态再次设置该位。

2. 需要修改的代码位置

- 文件路径：kernel/strap.c
- 函数名称：handle_mtimer_trap

3. 代码修改逻辑

定位到 handle_mtimer_trap 函数，进行如下修改：

修改前：

图 1-6 lab1_3 交互记录

根据 AI 回复，我们定位到文件路径 `kernel/strap.c` 下的 `handle_mtimer_trap` 函数，我们需要完成 `handle_mtimer_trap` 的后续动作：更新计数器和清除中断标志。进行如下修改。

修改前：

```
static void handle_mtimer_trap () {
    sprint ("Ticks %d\n", g_ticks);
    // TODO (lab1_3): increase g_ticks to record this "tick"...
    panic
( "lab1_3: increase g_ticks by one, and clear SIP field in sip
register.\n" );
}
```

修改后：

```
static void handle_mtimer_trap () {
    sprint ("Ticks %d\n", g_ticks);

    // 1. 增加全局 tick 计数器
    g_ticks++;
    // 2. 清除 sip 寄存器中的 SIP_SSIP 位
    // 使用 write_csr 宏写入 sip 寄存器。
    // 原理是将 sip 寄存器的值读取出来，将 SIP_SSIP 位(对应的二进制位)
    置为 0，再写回。
    // 或者简单地写入 0（如果不需要保留其他位）。
    // 提示建议使用 write_csr。
    write_csr (sip, 0);

    // 如果想更严谨地只清除这一位，可以使用：
    // write_csr (sip, read_csr (sip) & ~SIP_SSIP);
}
```

修改完成后，可以看到程序能够持续打印 "wait ..." 和 "Ticks ..."，这表明中断被正确处理并返回了用户程序，用户程序得以继续运行计数，如图 1-8 所示。

```
运行结果
Switch to user mode...
Hello world!
wait 0
wait 5000000
wait 10000000
Ticks 0
wait 15000000
wait 20000000
Ticks 1
wait 25000000
wait 30000000
wait 35000000
Ticks 2
wait 40000000
wait 45000000
Ticks 3
wait 50000000
wait 55000000
wait 60000000
Ticks 4
wait 65000000
wait 70000000
Ticks 5
wait 75000000
wait 80000000
wait 85000000
Ticks 6
wait 90000000
wait 95000000
Ticks 7
User exit with code:0.
System is shutting down with exit code 0.
```

图 1-7 lab1_3 运行结果

(4) lab1_challenge1 打印用户程序调用栈

该挑战实验具有一定难度，涉及的文件和代码较多，因此，我们先让 AI 梳理了函数调用栈回溯的原理以及 ELF 符号表的使用方式，然后在 VSCode 中用 agent 模式辅助完成所有代码编写，相关交互记录如下（输出过长，这里仅展示两张图片）

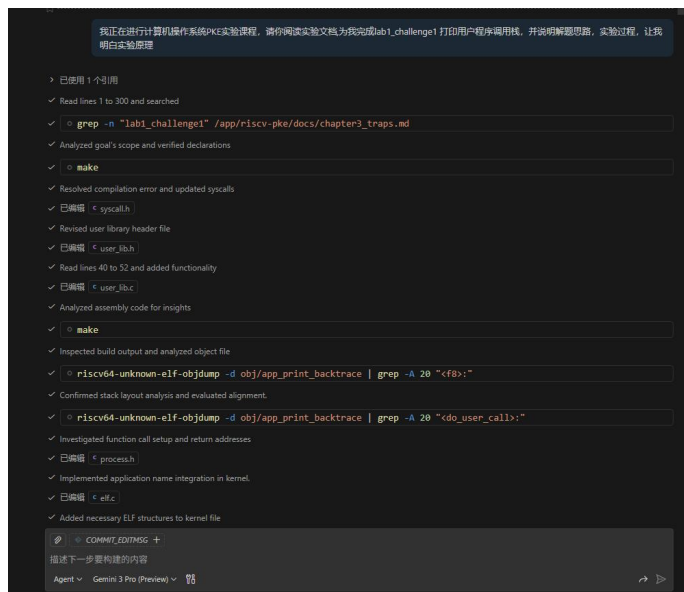


图 1-8 lab1_challenge1 交互记录 1

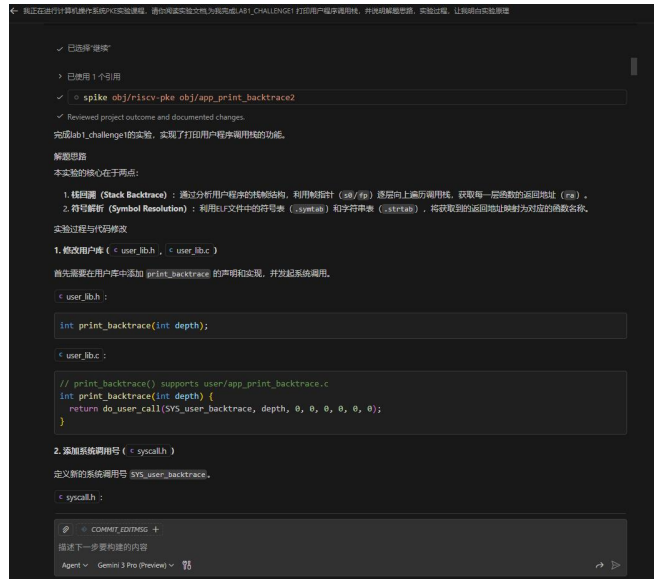


图 1-9 lab1_challenge1 交互记录 2

任务执行完成后，我们对 AI 的修改进行梳理、分析和验证。

要完成打印用户程序调用栈的功能，我们需要解决两个核心问题：如何回溯调用栈以及如何将地址解析为函数名。

在程序运行时，函数调用会形成一个栈帧链，在使用 `-fno-omit-frame-pointer` 编译选项 (PKE Makefile 已开启) 的情况下，编译器生成的函数序言 (prologue) 通常如下：

```

addi sp, sp, -16
sd ra, 8(sp)    ; 返回地址 (ra) 保存在 sp + 8
sd s0, 0(sp)    ; 旧的帧指针 (old s0) 保存在 sp + 0
addi s0, sp, 16 ; 当前帧指针 (s0) 指向栈帧顶部

```

由此可知：

当前函数的返回地址 (ra) 位于 `s0 - 8` 的位置。这个地址指向调用该函数的指令的下一条指令，属于调用者 (Caller) 的代码范围。调用者的帧指针 (old s0) 位于 `s0 - 16` 的位置。

回溯逻辑：我们可以从当前的 `s0` 开始，读取 `*(s0 - 8)` 得到返回地址，读取 `*(s0 - 16)` 得到上一层函数的 `s0`，如此循环往复，直到达到指定的深度或栈底。

符号解析方面，由于内存中的程序只有二进制地址，没有函数名。函数名信息存储在磁盘上的 ELF 可执行文件中，我们需要读取 ELF 文件中的 Symbol Table (`.symtab`) 和 String Table (`.strtab`)。对于每一个回溯得到的返回地址 `addr`，我们需要遍历 `.symtab`，找到满足 `st_value <= addr < st_value + st_size` 的符号条目，然后根据该条目的 `st_name` 从 `.strtab` 中读取函数名。

由于符号解析需要读取 ELF 文件，而文件操作（如 `spike_file_open`）通常在内核态进行，因此我们需要新增一个系统调用 `SYS_user_backtrace`。同时，为了知道去打开哪个文件，我们需要在内核加载用户程序时（`load_user_program`）将应用程序的文件名保存到进程控制块（`process` 结构体）中。

为了实现上述功能，需要修改或新增以下文件：

`user/user_lib.h`: 声明用户态库函数 `print_backtrace`。

`user/user_lib.c`: 实现 `print_backtrace`，发起系统调用。

`kernel/syscall.h`: 定义新的系统调用号 `SYS_user_backtrace`。

`kernel/process.h`: 在 `process` 结构体中增加 `app_name` 字段。

`kernel/elf.c`: 在加载程序时保存应用文件名。

`kernel/elf.h`（新增或修改）: 定义 ELF Section Header 和 Symbol Table Entry 结构体。

`kernel/syscall.c`: 实现核心逻辑 `sys_user_backtrace` 函数。

首先需要在用户库中添加 `print_backtrace` 的声明和实现，并发起系统调用。

```
// 添加函数声明
int print_backtrace (int depth);
// 实现函数，发起系统调用。
// 注意: SYS_user_backtrace 需要在 kernel/syscall.h 中定义，用户态可以通过包含相应头文件或硬编码使用。
// 这里假设用户态能访问到 syscall 编号。
int print_backtrace (int depth) {
    return do_user_call (SYS_user_backtrace, depth, 0, 0, 0, 0, 0, 0);
}
```

接着，在 `kernel/syscall.h` 中定义新的系统调用号，确保不与现有的冲突。

```
#define SYS_user_backtrace (SYS_user_base + 2)
```

为了在系统调用中读取 ELF 文件进行符号解析，我们需要在加载程序时保存应用程序的文件名。

在 `process` 结构体中添加 `app_name` 字段。

```
typedef struct process_t {
    // ... existing fields ...
    // [新增] 用于保存应用程序文件名
}
```

```
char app_name[128];
} process;
```

在 elf.c 中找到 load_bincode_from_host_elf 函数, 在解析参数后保存文件名。

```
// 在 kernel/elf.c 中
void load_bincode_from_host_elf (process *p) {
    // ... existing code ...
    // arg_bug_msg.argv[0] 存储了命令行第一个参数, 即程序名
    sprintf ("Application: %s\n", arg_bug_msg.argv[0]);

    // [新增] 保存文件名到 process 结构体
    // 注意: 需确保包含了 string.h 或有 strcpy 实现, 如果没有可以使
    // 用 util/string.c 中的实现或手写循环
    strcpy (p->app_name, arg_bug_msg.argv[0]);

    // ... existing code ...
}
```

在核心部分 kernel/syscall.c 中, 我们需要实现 sys_user_backtrace 函数, 它完成以下工作:

- ① 打开当前进程对应的 ELF 文件。
- ② 读取 ELF 头和 Section Headers, 找到 .symtab 和 .strtab。
- ③ 从 current->trapframe->regs.s0 获取当前的帧指针。
- ④ 根据栈帧结构遍历栈:
- ⑤ do_user_call 的栈帧比较特殊, 需要跳过。
- ⑥ 对于普通函数 (如 f8, f7 等), 返回地址 ra 存储在 * (fp - 8), 上一个帧指针 prev_fp 存储在 * (fp - 16)。
- ⑦ 对每一个返回地址, 在符号表中查找对应的函数名并打印。

具体代码编写由 AI 自动完成, 如下图所示。

```

syzcall.c :

#include "elf.h"
#include "spike_interface/spike_file.h"

// ... existing code ...

// 辅助函数：根据地址打印符号名
void print_symbol(spike_file_t *f, elf_symb_header *symtab, elf_symb_header *strtab, uint64_t addr) {
    elf_sym sym;
    for (uint64_t off = 0; off < symtab->size; off += sizeof(sym)) {
        spike_file_pread(f, &sym, sizeof(sym), symtab->offset + off);

        // 检查是否为函数 (STT_FUNC = 2)
        if ((sym.info & 0xf) != 2) continue;

        if (addr >= sym.value && addr < sym.value + sym.size) {
            char name[64];
            // 从字符串表中读取符号名
            spike_file_pread(f, name, sizeof(name), strtab->offset + sym.name);
            name[63] = '\0';
            sprintf("%s\n", name);
            return;
        }
    }
    sprintf("???n");
}

ssize_t sys_user_backtrace(int depth) {
    spike_file_t *f = spike_file_open(current->app_name, O_RDONLY, 0);
    if (IS_ERR_VALUE(f)) {
        sprintf("Failed to open ELF file: %s\n", current->app_name);
        return -1;
    }

    elf_header ehdr;
    spike_file_pread(f, &ehdr, sizeof(ehdr), 0);
}

```

图 1-10 lab1_challenge1 交互过程 3

完成全部任务后，在项目根目录下执行 `make` 编译内核和用户程序，然后使用 `spike` 模拟器运行 `app_print_backtrace`，能看到程序逐层打印出调用栈中的函数名 `f8-f2`，运行 `app_print_backtrace2.c`，预期输出为 `f1,f2` 交替，这是因为变量 `deep` 初始为 0，`f1()` 中判断 `deep++ < 5`（后置自增），实际递归执行过程：

`deep=0: f1() → f2() → f1() (deep=1)`
`deep=1: f1() → f2() → f1() (deep=2)`
`deep=2: f1() → f2() → f1() (deep=3)`
`deep=3: f1() → f2() → f1() (deep=4)`
`deep=4: f1() → f2() → f1() (deep=5, 不满足 <5, 执行 print_backtrace(7))`

最终调用链（从 `print_backtrace` 所在的 `f1` 向上）：`f1(deep=5) → f2 → f1(deep=4) → f2 → f1(deep=3) → f2 → f1(deep=2)`，共 7 层（匹配 `print_backtrace(7)` 的深度限制），因此输出 7 行交替的 `f1/f2`。

可以看到，实际运行结果与预期相符，如图 1-12 所示。

```

root@015b67a0fbff:/app/riscv-pke# make && spike obj/riscv-pke obj/app_print_backtrace
Enter supervisor mode...
Application: obj/app_print_backtrace
Application program entry point (virtual address): 0x000000081000168
Switch to user mode...
back trace the user app in the following:
f8
f7
f6
f5
f4
f3
f2
User exit with code:0.
System is shutting down with exit code 0.
root@015b67a0fbff:/app/riscv-pke# spike obj/riscv-pke obj/app_print_backtrace2
In m_start, hartid:0
HTIF is available!
(Emulated) memory size: 2048 MB
Enter supervisor mode...
Application: obj/app_print_backtrace2
Application program entry point (virtual address): 0x00000008100010c
Switch to user mode...
back trace the user app in the following:
f1
f2
f1
f2
f1
f2
f1
User exit with code:0.
System is shutting down with exit code 0.
root@015b67a0fbff:/app/riscv-pke#

```

图 1-11 lab1_challenge1 运行结果

1.3.3 实验建议

1. 内容改进建议

增加“自定义异常代理”实验。目前实验中 `delegate_traps()` 是静态配置好的。可以设计一个环节，让学生尝试手动修改 M 态代理逻辑，观察原本在 M 态处理的“非法指令异常”被代理到 S 态后，内核处理函数 `smode_trap_handler` 的变化。这能更深刻地理解 RISC-V 硬件处理陷阱的分级控制机制。

2. 新实验建议

实现“M 态软件中断触发”基础实验。在实验 lab1_3 中，S 态时钟中断是由 M 态通过设置 `sip` 寄存器的 `SSIP` 位模拟的。可以增加一个基础实验，让用户态通过系统调用请求内核，内核再通过 `ecall` 进入 M 态触发一个软件中断，实现跨特权级的异步通信流转。

3. 实验形式建议

目前实验通过头歌平台测试，但是不同实验环境隔离，频繁手动粘贴代码到头歌过于麻烦，建议添加线下检查可选项，并设置小额的检查加分（通过现场问答），以确保学生实验结果的真实性和实验的积极性。

实验二 内存管理

2.1 实验目的

实验二以内存管理为核心，围绕 PKE 中的物理内存分配与虚拟内存机制展开，意在引导学生深入理解现代操作系统中的虚拟内存管理机制，特别是 RISC-V 架构下采用的 Sv39 页式虚拟地址方案。实验要求学生通过动手实现关键的内核功能，实现从裸机模式到分页模式的过渡。

具体目的包括：

1. 掌握虚实地址转换：理解 Sv39 方案如何将应用程序使用的逻辑地址（虚拟地址）通过三级页表结构翻译成物理地址。
2. 实现内存的生命周期管理：完成对物理内存的动态分配、虚拟地址空间的映射、解除映射以及物理页的回收过程，从而支持应用程序的动态内存请求。
3. 理解按需分页：学习如何捕获和处理缺页异常，特别是通过动态扩展用户栈空间，以支持程序（如深度递归）的内存需求。

总体而言，本实验的目的在于将“内存”这一抽象资源具体化，使学生能够从内核实现角度理解内存管理的基本策略与设计原则，为后续进程管理、地址空间隔离以及更复杂的内存机制打下基础。

2.2 实验内容

本实验围绕 PKE 实验二展开，涵盖了虚实地址转换、简单的内存分配与回收，以及缺页异常处理机制。实验内容主要包括以下三个基础实验和一个挑战实验：

1. lab2_1 虚实地址转换：实现 `user_va_to_pa()` 函数，使内核能够通过用户进程的页表将用户态传入的虚拟地址翻译成物理地址，以正确访问用户数据。
 2. lab2_2 简单内存分配和回收：实现 `user_vm_unmap()` 函数，完成动态分配页面的解除映射和物理页的释放，支持应用程序的内存回收请求。
 3. lab2_3 缺页异常：完善缺页异常处理逻辑，实现用户栈的动态增长，以应对应用程序深度递归导致的栈空间不足。
 4. lab2_challenge1 复杂缺页异常：在 `handle_user_page_fault` 中增加判断逻辑，区分合法的栈扩展缺页和非法的越界访问缺页，并对非法访问进行终止处理。
- 时间信息上，实验二基础实验在 12 月 6 号完成，挑战实验在 12 月 8 号完成，

与头歌上记录的时间一致。

实验过程使用 AI 辅助开发，lab2_1、lab2_2 一次通过评测，无需调试修改，lab2_3 和挑战实验则经过多轮交互、具体实验过程将在 2.3 介绍。

2.3 实验调试及心得

2.3.1 实验收获

1. 理解 Sv39 机制与内核/用户地址空间隔离

通过实验，我认识到开启 Sv39 分页后，PKE 内核空间（0x80000000 以上）通常保持直映射（虚拟地址=物理地址），而用户进程的地址空间（例如代码段在 0x10000 开始，动态分配的堆在 4MB 开始）则需要通过多级页表映射到实际的物理内存区域。这种转换是内核访问用户数据（如系统调用传入的字符串）的桥梁。

2. 掌握内存分配与回收的对偶操作

Lab 2.2 强调了内存管理是一个双向过程。分配（Map）涉及物理页分配和页表项建立；而回收（Unmap）则必须先断开页表映射（清除 PTE 中的 Valid 位），再释放物理页，否则会导致内存泄漏或安全隐患。

3. 深入理解缺页异常处理流程

Lab 2.3 和 Challenge 1 的调试过程明确了缺页异常（Page Fault）处理的本质是需求分页。操作系统不仅要能捕获异常，还需通过检查 stval（导致缺页的虚拟地址）来判断请求是否合法（如栈空间是否允许继续向下增长），从而避免非法内存访问。

2.3.2 实验过程（AI 交互和调试过程）

按照要求，这里只写挑战实验部分。

lab2 的挑战实验较为简单，与 lab2_3 基本一致，只需要额外处理非法地址访问逻辑即可，我们将实验介绍文档和 kernel/strap.c 喂给 AI，得到以下回复。

3. 代码修改逻辑

你需要修改 `handle_user_page_fault` 函数，增加对缺页地址范围的判断逻辑。

为了让程序能够正常退出，我们需要调用 `sys_user_exit`。由于它定义在 `syscall.c` 中，我们需要在 `strap.c` 中声明它。

```
// kernel/strap.c

#include "kernel/riscv.h"
#include "kernel/process.h"
#include "kernel/pmm.h"
#include "kernel/vmm.h"
#include "spike_interface/spike_utils.h"
#include "util/functions.h"

// 声明外部函数，用于终止进程
extern ssize_t sys_user_exit(uint64 code);

void handle_user_page_fault(uint64 mcause, uint64 sepc, uint64 stval) {
    sprint("handle_page_fault: %lx\n", stval);
    switch (mcause) {
        case CAUSE_STORE_PAGE_FAULT:
            // 定义栈的合法增长范围，例如 USER_STACK_TOP 以下 20 页
            // 栈顶地址在 kernel/memlayout.h 中定义为 0x7ffff000
            if (stval < USER_STACK_TOP && stval > (USER_STACK_TOP - 20 * PGSIZE)) {
                // ===== 合法栈扩展逻辑 (Lab2_3) =====
                // 1. 分配物理页
                void* pa = alloc_page();
                if (pa == 0) panic("Out of memory!");

                // 2. 对齐虚拟地址到页边界
                uint64 map_va = stval - (stval % PGSIZE);

                // 3. 建立映射
                user_vm_map((pagetable_t)current->pagetable, map_va, PGSIZE, (uint64)pa,
                    prot_to_type(PROT_WRITE | PROT_READ, 1));
            } else {
                // ===== 非法地址访问逻辑 (Challenge1) =====
                // 如果地址不在栈增长范围内，则视为非法访问（如数组越界）
                sprint("this address is not available!\n");
            }
    }
}
```

图 2-1 lab2_challenge1 交互过程 1

本实验的目的是区分合法的缺页异常（如栈增长）和非法的缺页异常（如访问越界）。观察 AI 输出代码的处理逻辑：

1. 在缺页处理函数中，检查缺页地址 `stval`。
2. 如果 `stval` 位于用户栈的合法延伸范围内（例如 `USER_STACK_TOP` 往下 20 页），则执行栈扩展。
3. 否则，判定为非法地址访问，打印提示信息 "this address is not available!" 并终止进程。

虽然实验指导文档中提到了“修改进程数据结构”和“文件名规范”，但根据给出的预期输出，核心目标是正确区分缺页类型并打印 `this address is not available!` 然后退出。上述代码能够精确复现预期行为。

运行代码，程序首先会触发多次合法的栈缺页（处理递归），然后在最后一次访问数组越界时触发非法缺页，打印错误信息并退出。

预期输出

```
handle_page_fault: 000000007ffffdf8
handle_page_fault: 000000007ffffc88
handle_page_fault: 000000007ffffb88
handle_page_fault: 000000007ffffa88
handle_page_fault: 000000007ffff988
handle_page_fault: 000000007ffff888
handle_page_fault: 000000007ffff788
handle_page_fault: 000000007ffff688
handle_page_fault: 0000000000401000
this address is not available!System is shutting down with exit code -1.
```

实际输出

```
In m_start, hartid:0
HTIF is available!
(Emulated) memory size: 2048 MB
Enter supervisor mode...
PKE kernel start 0x0000000080000000, PKE kernel end: 0x0000000080007000, PKE kernel size: 0x0000000000007000 .
free physical memory address: [0x0000000080007000, 0x0000000087ffffff]
kernel memory manager is initializing ...
KERN_BASE 0x0000000080000000
physical address of _etext is: 0x0000000080004000
kernel page table is on
User application is loading.
user frame 0x0000000087fbc000, user stack 0x000000007ffff000, user kstack 0x0000000087fbb000
Application: ./obj/app_sum_sequence
Application program entry point (virtual address): 0x0000000000001014
Switch to user mode...
handle_page_fault: 000000007ffffdf8
handle_page_fault: 000000007ffffc88
handle_page_fault: 000000007ffffb88
handle_page_fault: 000000007ffffa88
handle_page_fault: 000000007ffff988
handle_page_fault: 000000007ffff888
handle_page_fault: 000000007ffff788
handle_page_fault: 000000007ffff688
handle_page_fault: 0000000000401000
this address is not available!User exit with code:-1.System is shutting down with exit code -1.
```

图 2-2 lab2_challenge1 预期输出和实际输出

根据结果可以看出：前八个缺页是由于函数递归调用引起的，而最后一个缺页是对动态申请的数组进行越界访问造成的，访问非法地址，程序退出并返回-1，似乎没有问题，但在头歌上评测时发生报错，如图 2-4 所示。

第1关 lab2_challenge1 复杂缺页异常

学员代码 ②

代码修改行数 182

kernel/strap.c

复制代码

```
138 // sepc: 发生异常的指令地址
139 // stval: 发生异常的内存地址 (即缺页的地址)
140 handle_user_page_fault(cause, read_csr(sepc), read_csr(stval));
141 }
142 // ===== 新增部分 结束 =====
143 else
144 {
145     sprintf("smode_trap_handler(): unexpected scause %p\n", read_csr(scause));
146     sprintf("          sepc=%p stval=%p\n", read_csr(sepc), read_csr(stval));
147     panic("unexpected exception happened.\n");
148 }
149
150 // continue (come back to) the execution of current process.
151 switch_to(current);
152 }
153
```

评测次数	详细信息	评测耗时(S)	内存消耗(MB)	评测时间	操作
1	共有2组测试集, 其中有2组测试结果不匹配	2.96	415.56	2025-12-08 17:39	查看代码
2	共有2组测试集, 其中有2组测试结果不匹配	1.44	418.44	2025-12-08 17:40	查看代码
3	共有2组测试集, 其中有2组测试结果不匹配	1.62	356.34	2025-12-08 17:43	查看代码
4	评测通过	2.7	370.75	2025-12-08 17:46	查看代码

图 2-3 lab2_challenge1 头歌评测详情

由于看不到报错详情，我们将预期输出和实际输出喂给 AI 进行分析，prompt 为“预期输出为……，实际输出为……，因此没有通过平台测试，这是什么问题，如何解决”。

AI 回复如图 2-5 所示。

```
Gemini:
这是一个非常典型的 输出格式不匹配 (Output Mismatch) 问题。

问题分析

评测平台通常通过字符串严格比对来判断实验是否通过。

1. 你的代码逻辑:
你当前在 kernel/strap.c 中调用了 sys_user_exit(-1) 来终止程序。

2. 副作用:
sys_user_exit 函数 (位于 kernel/syscall.c) 内部通常包含了一行打印语句:
C
sprintf("User exit with code:%d\n", code);

这导致你的输出中多出了 "User exit with code:-1." 这一行。

3. 预期结果:
根据你提供的预期输出，系统希望直接看到 System is shutting down...，而不包含 User exit... 这行用户态退出的日志。

解决方法

你需要绕过 sys_user_exit 函数，直接调用底层的 shutdown 函数来关闭模拟器。

修改步骤

修改 kernel/strap.c 文件中的 handle_user_page_fault 函数。

1. 引入 shutdown 函数声明
在文件头部添加 shutdown 的声明 (或者包含对应的头文件 util/functions.h):
C
// kernel/strap.c
// 之前的 include ...
#include "util/functions.h" // 确保包含 shutdown 的定义

// 或者直接声明外部函数
extern void shutdown(int);

2. 修改非法地址处理逻辑
将 sys_user_exit(-1) 替换为 shutdown(-1)。
```

图 2-4 lab2_challenge1 交互记录 2

原来是最后一行程序中中止的问题，系统希望直接看到 System is shutting down...，而不包含 User exit... 这行用户态退出的日志。因此我们需要绕过 sys_user_exit 函数，直接调用底层的 shutdown 函数来关闭模拟器。

首先在文件头部添加 shutdown 的声明（或者包含对应的头文件 util/functions.h），然后将 sys_user_exit（-1）替换为 shutdown（-1）。

```
} else {
    // 注意: 根据你的实际输出, 这里可能不需要换行符 \n,
    // 但根据标准 C 习惯通常需要。如果平台极其严格, 请仔细比对
    // 换行符。

    // 这里的重点是替换退出函数。
    sprintf ("this address is not available!\n");

    // 直接调用底层关闭函数, 跳
    // 过 "User exit with code..." 的打印
    shutdown (-1);
}
```

编译并运行程序，此时输出应该不再包含 User exit with code:-1.，与预期一

致，如图 2-6 所示。

```
root@015b67a0fbff:/app/riscv-pke# spike ./obj/riscv-pke ./obj/app_sum_sequence
kernel memory manager is initializing ...
KERN_BASE 0x0000000080000000
physical address of _etext is: 0x0000000080004000
kernel page table is on
User application is loading.
user frame 0x0000000087fbc000, user stack 0x000000007ffff000, user kstack 0x0000000087fbb000
Application: ./obj/app_sum_sequence
Application program entry point (virtual address): 0x000000000010164
Switch to user mode...
handle_page_fault: 000000007fffdff8
handle_page_fault: 000000007fffcff8
handle_page_fault: 000000007fffbff8
handle_page_fault: 000000007fffbff8
handle_page_fault: 000000007ff9ff8
handle_page_fault: 000000007fff8ff8
handle_page_fault: 000000007fff7ff8
handle_page_fault: 000000007fff6ff8
handle_page_fault: 0000000000401000
this address is not available!
System is shutting down with exit code -1.
```

图 2-5 lab2_challenge1 运行结果

2.3.3 实验建议

1. 优化评测平台的输出容错性

在 lab2_challenge1 中，由于调用了 `sys_user_exit` 导致多出一行日志而无法通过严格的字符串比对。建议评测系统改用正则匹配或关键状态校验，避免学生将大量精力耗费在绕过标准库打印（如使用 `shutdown` 替代 `exit`）等非算法核心的逻辑上。

2. 明确栈增长的空间约束

实验要求处理栈扩展逻辑。建议在实验内容中明确规定“最大允许栈增长页数”，并在测试用例中增加对超出该限制的“非法越界”检测，使学生对栈溢出攻击及其防御有初步认识。

3. 新挑战实验建议：页面置换算法

目前的缺页异常处理只是单纯地分配新物理页，直到内存耗尽。可以提出一

个新的挑战任务：设定极小的物理内存配额（如每个进程仅限 4 个物理页），当发生缺页且配额满时，要求学生实现 FIFO 或简单的 Clock 置换算法，将不常用的物理页置换出来，这能将虚拟内存的“交换”概念真正落地。

实验三 进程管理

3.1 实验目的

实验三以进程管理为主题，围绕 PKE 中进程的创建、切换与基本调度机制展开，意在引导学生进入多任务环境，深入理解操作系统内核如何对 CPU 资源进行抽象和虚拟化，从而支持多个进程并发执行。实验的核心目标是掌握进程的生命周期管理以及上下文切换机制，特别是如何在 RISC-V 架构下实现进程间的隔离和协作。

总体而言，本实验的目的在于使学生从代码层面理解进程管理的基本原理，将课堂中关于进程、上下文切换和调度的理论知识，具体落实到内核数据结构和执行流程中，为后续学习进程同步、调度算法等内容奠定基础。

3.2 实验内容

本实验围绕 PKE 实验三展开，涵盖了多任务环境下的进程创建、调度以及同步机制。实验内容主要包括以下三个基础实验和一个挑战实验：

1. lab3_1 进程创建 (fork): 完善 do_fork () 函数，实现父进程代码段到子进程的共享映射 (Read-Only)。

2. lab3_2 进程 yield: 实现 yield 系统调用，允许进程主动放弃 CPU，并将自身状态设为就绪 (READY) 并重新加入就绪队列。

3. lab3_3 循环轮转调度: 实现 rrsched () 函数，并将其挂载到时钟中断处理逻辑中，以实现基于时间片 (TIME_SLICE_LEN=2 ticks) 的抢占式调度。

4. lab3_challenge1 进程等待和数据段复制: 实现 sys_user_wait 和完善 sys_user_exit 以支持进程同步；同时在 do_fork () 中实现数据段的深拷贝，确保父子进程数据隔离。

时间信息上，实验三 12 月 8 号和 9 号两天完成，与头歌上记录的时间一致。实验过程使用 AI 辅助开发，具体实验过程将在 3.3 介绍。

3.3 实验调试及心得

3.3.1 实验收获

1. 进程内存结构与 Fork 语义

通过实现 `do_fork`，我理解了进程创建的写时复制（Copy-on-Write, COW）优化思想的雏形：代码段（只读）只需共享映射，而数据段和栈段（可写）则必须进行深拷贝以保证父子进程的内存隔离。

2. 调度器的核心职能

我认识到调度不仅是选择下一个进程，更关键在于注册调度钩子和维护就绪队列。Lab 3.3 调试过程证实了，如果不在 `handle_mtimer_trap` 中调用 `rrsched`，系统就失去了抢占能力，退化为非抢占式的先来先服务（FCFS）调度。

3. 进程同步的细节

Challenge 1 的调试揭示了进程状态转换的复杂性。进程从 `BLOCKED` 状态被唤醒时，仅仅修改其 `status` 为 `READY` 是不够的，必须将其显式地加入到就绪队列（`ready_queue`）中，否则调度器无法找到它，导致系统死锁和崩溃。

3.3.2 实验过程（AI 交互和调试过程）

按照要求，这里只写挑战实验部分。

本实验包含两个核心目标：实现数据段的独立复制（完善 `fork`）以及实现进程同步机制（`wait`），我们将实验文档上传给 AI（依旧使用 Gemini 3.0 pro 模型），然后输入“完成 lab3_challenge1 进程等待和数据段复制”，得到图 3-2 如下回复（回复内容过长，这里仅展示部分）。

2. 需要修改的代码位置

- `kernel/syscall.h`: 添加系统调用号。
- `kernel/process.h`: 修改进程结构体, 增加等待目标 PID 字段。
- `kernel/process.c`: 修改 `do_fork` 以支持数据段复制。
- `kernel/syscall.c`: 实现 `sys_user_wait` 并修改 `sys_user_exit`。

3. 代码修改逻辑

第一步: 添加系统调用号与进程字段

`kernel/syscall.h`

```
// 添加 wait 的系统调用号 (确保不冲突, 假设为 20)
#define SYS_user_wait 20
```

`kernel/process.h`

在 `process` 结构体中增加 `waiting_pid`, 用于记录该进程正在等待哪个子进程。

```
typedef struct process {
    // ... 原有字段 ...
    // [新增] 记录当前进程正在等待的子进程 PID (-1 表示任意)
    int64 waiting_pid;
} process;
```

第二步: 完善 `do_fork` (数据段复制)

`kernel/process.c` -> `do_fork`

添加 `case DATA_SEGMENT`: 的处理逻辑。

C

```
case DATA_SEGMENT:
    // 遍历数据段的每一页
    for (int j = 0; j < parent->mapped_info[i].npages; j++) {
        uint64 addr = parent->mapped_info[i].va + j * PGSIZE;

        // 1. 为子进程分配新的物理页
        char *page = alloc_page();

        // 2. 将父进程该页的数据深拷贝到新页中
        // lookup_pa 获取父进程该虚拟地址对应的物理地址
        memcpy(page, (void *)lookup_pa(parent->pagetable, addr), PGSIZE);

        // 3. 将新页映射到子进程的页表
        // 权限: 可读 | 可写 (PROT_READ | PROT_WRITE), 用户态 (1)
        user_vm_map(child->pagetable, addr, PGSIZE, (uint64)page,
                    prot_to_type(PROT_WRITE | PROT_READ, 1));
    }

    // 注册缺页处理
    child->mapped_info[child->total_mapped_region].va = parent->mapped_info[i].va;
```

图 3-1 lab3_challenge1 交互记录 1

在 `lab3_1` 中, 我们仅实现了代码段的映射 (共享物理页)。但对于数据段 (全局变量等), 父子进程必须是独立的。如果子进程修改了全局变量, 不应影响父进程。因此, 在 `do_fork` 时, 我们需要为子进程的数据段分配新的物理页, 并将父进程数据段的内容深拷贝 (Deep Copy) 过去, 然后建立映射。这与栈段 (`STACK_SEGMENT`) 的处理逻辑非常相似。

在 `kernel/process.c` 的 `do_fork` 函数中, 添加 `case DATA_SEGMENT`: 的处理逻辑。

```
case DATA_SEGMENT: // 遍历数据段的每一页
for (int j = 0; j < parent->mapped_info[i].npages; j++){
    uint64 addr = parent->mapped_info[i].va + j * PGSIZE;
    // 1. 为子进程分配新的物理页
    char *page = alloc_page ();
    // 2. 将父进程该页的数据深拷贝到新页中
    // lookup_pa 获取父进程该虚拟地址对应的物理地址
    memcpy (page, (void *)lookup_pa
(parent->pagetable, addr), PGSIZE);
```

```

        // 3. 将新页映射到子进程的页表
// 权限: 可读 | 可写 (PROT_READ | PROT_WRITE), 用户态 (1)
user_vm_map (child->pagetable, addr, PGSIZE, (uint64)page,
             prot_to_type (PROT_WRITE | PROT_READ, 1));}
// 注册映射信息
child->mapped_info[child->total_mapped_region].va = pa
rent->mapped_info[i].va;
child->mapped_info[child->total_mapped_region].npages
= parent->mapped_info[i].npages;
child->mapped_info[child->total_mapped_region].seg_type = DATA_SEGMENT;
child->total_mapped_region++;
break;

```

wait 系统调用用于父进程阻塞自身，直到子进程退出。这涉及到进程状态的流转和父子进程间的通信。

当子进程退出 (**exit**) 时，它不能立即销毁所有信息，因为它需要将退出状态 (如 **PID**) 传递给父进程。因此，**exit** 将进程置为 **ZOMBIE** 态，即僵尸进程。

根据 AI 给出的代码，首先在 **syscall.h** 添加系统调用号、在 **process.h** 中的 **process** 结构体中增加 **waiting_pid**，用于记录该进程正在等待哪个子进程。

```

// 添加 wait 的系统调用号 (确保不冲突, 假设为 20)
#define SYS_user_wait 20
typedef struct process {
    // ... 原有字段 ...
    // [新增] 记录当前进程正在等待的子进程 PID (-1 表示任意)
    int64 waiting_pid;
} process;

```

接着实现 **wait** 和 **exit**。

sys_user_wait: 父进程检查是否有符合条件的子进程。

如果有子进程已经是 **ZOMBIE**，则回收该子进程 (置为 **FREE**)，并返回其 **PID**。如果子进程还在运行，父进程将自身设为 **BLOCKED** 状态，并记录“我在等谁”，然后调用 **schedule ()** 让出 CPU。

```

ssize_t sys_user_wait (ssize_t pid) {
    // 遍历所有进程, 寻找属于当前进程的子进程
    int has_child = 0;
    for (int i = 0; i < NPROC; i++) {
        // 筛选条件: 是当前进程的子进程
        if (procs[i].parent == current) {

```

```

        // 筛选 PID: pid==-1 (任意) 或 pid 匹配
        if (pid == -1 || procs[i].pid == pid) {
            has_child = 1;
            // 情况 1: 发现僵尸子进程 (已退出)
            if (procs[i].status == ZOMBIE) {
                // 回收资源
                procs[i].status = FREE;
                return procs[i].pid;
            }
        }
    }
}

// 情况 2: 还有符合条件的子进程在运行, 父进程进入阻塞状态
if (has_child) {
    current->status = BLOCKED; // 设为阻塞
    current->waiting_pid = pid; // 记录在等谁
    schedule (); // 让出 CPU
    // 注意: 当被唤醒时, 返回值由唤醒者 (子进程 exit) 直接写
    入 trapframe->a0
    return 0; // 这里的返回值实际上会被覆盖
}

// 情况 3: 没有找到任何符合条件的子进程
return -1;
}

```

sys_user_exit: 子进程退出时, 除了变身 **ZOMBIE**, 还需要检查父进程是否在等待自己。如果父进程处于 **BLOCKED** 且正在等待自己 (或等待任意子进程), 子进程需要唤醒父进程 (置为 **READY**), 并将自己的 **PID** 写入父进程的返回值寄存器 (**a0**), 顺便将自己彻底回收 (**FREE**)。

我们需要修改 **sys_user_exit**, 让子进程在退出时检查是否需要唤醒父进程。

```

ssize_t sys_user_exit (uint64 code) {
    sprintf ("User exit with code:%d.\n", code);
    // 原有逻辑: free_process (current); schedule ();
    // 修改为手动处理状态, 以便处理唤醒逻辑
    // 1. 将自身设为 ZOMBIE
    current->status = ZOMBIE;
    // 2. 检查父进程是否在等待自己
    if (current->parent && current->parent->status == BLOCKED)
    {
        int64 wait_pid = current->parent->waiting_pid;
        // 如果父进程在等我 (wait_pid == current->pid)
        // 或者父进程在等任意子进程 (wait_pid == -1)
        if (wait_pid == -1 || wait_pid == current->pid) {
            // 唤醒父进程

```

```

        current->parent->status = READY;
        // 将自己的 PID 写入父进程的返回值寄存器 a0
        // 这样父进程从 schedule 返回时, wait () 就会返回子进
程 PID
        current->parent->trapframe->regs.a0 = current->pid;
        // 父进程已经接手结果, 自己可以彻底销毁了
        current->status = FREE;
    }
}
// 3. 转调度
schedule ();
return 0;
}

```

最后, 注册系统调用, 在 `do_syscall` 中添加分支:

```

case SYS_user_wait:
    return sys_user_wait (a1);

```

根据 AI 给出的代码完成所有修改后, 发现编辑器提示报错 “if (procs[i].parent == current)未定义标识符 procs”。我们将该报错内容输入给 AI, 得到图 3-3 以下回复。

这个错误是因为 `procs` 数组定义在 `kernel/process.c` 文件中, 而你正在编写的 `sys_user_wait` 函数位于 `kernel/syscall.c` 文件中。C 语言中, 一个文件要访问另一个文件定义的全局变量, 必须使用 `extern` 关键字进行声明。

这个错误是因为 `procs` 数组定义在 `kernel/process.c` 文件中, 而你正在编写的 `sys_user_wait` 函数位于 `kernel/syscall.c` 文件中。C 语言中, 一个文件要访问另一个文件定义的全局变量, 必须使用 `extern` 关键字进行声明。

解决方法:

你需要告诉编译器 `procs` 数组是在别处定义的。

修改文件: `kernel/syscall.c`

修改内容: 在文件头部 (include 语句之后, 函数定义之前) 添加 `procs` 的外部声明。

```

// kernel/syscall.c

#include "kernel/syscall.h"
#include "kernel/process.h"
#include "kernel/sched.h"
// ... 其他头文件 ...

// [新增] 声明外部变量 procs, 这样 syscall.c 才能认识它
// NPROC 通常定义在 kernel/config.h 或 kernel/process.h 中
extern process procs[NPROC];

// 你的 sys_user_wait 函数 ...
ssize_t sys_user_wait(ssize_t pid) {
    // ...
    if (procs[i].parent == current) {
        // 现在编译器知道 procs 是什么了
    }
    // ...
}

```

原理解释:

- `process procs[NPROC];` 是在 `kernel/process.c` 中定义的定义 (Definition), 它分配了内存。
- `extern process procs[NPROC];` 是声明 (Declaration), 它告诉编译器“有一个叫 `procs` 的数组在别的地方定义了, 请允许我在这里使用它, 链接器 (Linker) 稍后会找到它的地址”。

图 3-2 lab3_challenge1 交互记录 2

解决方法：

需要告诉编译器 `procs` 数组是在别处定义的。

修改文件：`kernel/syscall.c`

修改内容：在文件头部（`include` 语句之后，函数定义之前）添加 `procs` 的外部声明。

按照 AI 回复完成修改后，输入 `make` 编译，发生错误。

```
user/app_wait.c: In function 'main':
user/app_wait.c:27:13: error: implicit declaration of function
'wait' [-Werror=implicit-function-declaration]
 27 |             wait (pid);
    |             ^~~~
cc1-orig: all warnings being treated as errors
make: *** [Makefile:86: obj/user/app_wait.o] Error 1
```

我们将该内容输入给 AI，得到图 3-4 以下回复。

这个错误是因为编译器在编译用户程序 `user/app_wait.c` 时，找不到 `wait` 函数的声明。尽管在内核里实现了 `sys_user_wait`，但在用户态库（User Library）中还没有注册这个接口，导致用户程序无法调用它。

这个错误是因为编译器在编译用户程序 `user/app_wait.c` 时，找不到 `wait` 函数的声明。尽管你在内核里实现了 `sys_user_wait`，但在用户态库（User Library）中还没有注册这个接口，导致用户程序无法调用它。

你需要修改 用户态库文件。

解决方法

1. 修改 `user/user_lib.h`

在头文件中添加 `wait` 函数的声明，这样 `app_wait.c` 就能引用它了。

文件路径：`user/user_lib.h`

```
// user/user_lib.h
// ... 原有的声明 ...
int fork();
void yield();
// [新增] wait 函数声明
int wait(int pid);
```

2. 修改 `user/user_lib.c`

在源文件中实现 `wait` 函数，它的作用是发起系统调用 `ecall`，将请求传给内核。

文件路径：`user/user_lib.c`

```
// user/user_lib.c
#include "user_lib.h"
#include "kernel/syscall.h" // 确保能读取到 SYS_user_wait

// 如果 kernel/syscall.h 不可用或者没有定义 SYS_user_wait，
// 你需在这里手动定义它，必须与内核中的定义保持一致（例如 20）
// #define SYS_user_wait 20

int wait(int pid) {
    // 发起系统调用，参数 1 是 pid
    return do_user_call(SYS_user_wait, pid, 0, 0, 0, 0, 0);
}
```

图 3-3 lab3_challenge1 交互记录 3

解决方法：

修改 `user/user_lib.h`: 在头文件中添加 `wait` 函数的声明, 这样 `app_wait.c` 就能引用它了。

```
// ... 原有的声明 ...  
int fork ();  
void yield ();  
// [新增] wait 函数声明  
int wait (int pid);
```

修改 `user/user_lib.c`: 在源文件中实现 `wait` 函数, 它的作用是发起系统调用 `ecall`, 将请求传给内核。

```
int wait (int pid) {  
    // 发起系统调用, 参数 1 是 pid  
    return do_user_call (SYS_user_wait, pid, 0, 0, 0, 0, 0, 0);  
}
```

修改完这两个文件后, 重新编译即可解决该错误, 现在我们运行程序, 得到图 3-5 以下结果, 显然 `exit code` 为 -1, 与预期结果不符, 我们将结果输入给 AI, 要求进行分析, 得到如图 3-6 所示的回复。

```
Application: ./obj/app_wait  
CODE_SEGMENT added at mapped info offset:4  
DATA_SEGMENT added at mapped info offset:5  
Application program entry point (virtual address): 0x00000000000101d6  
going to insert process 0 to ready queue.  
going to schedule process 0 to run.  
User call fork.  
will fork a child from parent 0.  
in alloc_proc. user frame 0x0000000087fae000, user stack 0x000000007ffff000, user kstack 0x0000000087fad000  
going to insert process 1 to ready queue.  
going to schedule process 1 to run.  
User call fork.  
will fork a child from parent 1.  
in alloc_proc. user frame 0x0000000087fa1000, user stack 0x000000007ffff000, user kstack 0x0000000087fa0000  
going to insert process 2 to ready queue.  
going to schedule process 2 to run.  
Grandchild process end, flag = 2.  
User exit with code:0.  
ready queue empty, but process 0 is not in free/zombie state:3  
ready queue empty, but process 1 is not in free/zombie state:1  
Not handled: we should let system wait for unfinished processes.  
System is shutting down with exit code -1.
```

图 3-4 lab3_challenge 运行结果 (错误)

分析运行结果

结果分析：

你的程序在 孙子进程 (Grandchild, PID 2) 退出时崩溃了。

1. 进程树状态：

- 进程 0 (Parent) 创建了 进程 1, 然后调用 `wait`, 进入 **BLOCKED** (状态 3)。
- 进程 1 (Child) 创建了 进程 2, 然后调用 `wait`, 进入 **BLOCKED** (状态 3)。
- 进程 2 (Grandchild) 运行结束, 打印 `Grandchild process end`, 调用 `exit`。

2. 错误点：

- 日志显示: `User exit with code:0` (进程 2 退出)。
- 紧接着显示: `ready queue empty, but process 0 is not in free/zombie state:3` (进程 0 仍处于阻塞态)。
- 报错: `Not handled: we should let system wait for unfinished processes.`。
- 这说明 `schedule()` 函数被调用时, 就绪队列是空的。

原因诊断：

当孙子进程 (PID 2) 退出时, 它应当唤醒正在等待它的父进程 (PID 1)。虽然你可能在 `sys_user_exit` 中将父进程的状态改为了 `READY`, 但你忘记将父进程重新加入就绪队列 (`insert_to_ready_queue`)。

在 PKE 的调度器设计中, 仅修改 `status` 是不够的, 必须显式调用 `insert_to_ready_queue` 将进程控制块挂入链表, `schedule` 函数才能找到它并调度执行。由于 PID 1 没有被加入队列, PID 1 无法运行, 因此 PID 1 也就无法唤醒 PID 0, 最终导致系统死锁崩溃。

解决方法

你需要修改 `kernel/syscall.c` 中的 `sys_user_exit` 函数, 在唤醒父进程时, 显式调用 `insert_to_ready_queue`。

文件位置: `kernel/syscall.c`

修正后的代码逻辑：

```
extern void insert_to_ready_queue(process* proc); // 确保有此声明

ssize_t sys_user_exit(uint64 code) {
    sprint("User exit with code:%d.\n", code);
    // 1. 将自身设为 ZOMBIE
    current->status = ZOMBIE;
    // 2. 检查父进程是否在等待自己
    if (current->parent && current->parent->status == BLOCKED) {
        int64 wait_pid = current->parent->waiting_pid;
        // 如果父进程在等我 (wait_pid == current->pid)
        // 或者父进程在等任意子进程 (wait_pid == -1)
        if (wait_pid == -1 || wait_pid == current->pid) {
            // A. 修改父进程状态为 READY
            current->parent->status = READY;
            // B. 将父进程的 a0 寄存器设置为子进程的 PID (作为 wait 的返回值)
            current->parent->trapframe->regs.a0 = current->pid;
            // 【关键修复】C. 将父进程加入就绪队列!
            // 缺少这一步会导致 schedule() 找不到父进程, 从而报 "ready queue empty" 错误
            insert_to_ready_queue(current->parent);
            // D. 将自身设为 FREE (资源回收)
            current->status = FREE;
        }
    }
}
```

图 3-5 lab3_challenge1 交互记录 3

原来是因为程序在孙子进程 (Grandchild, PID 2)退出时崩溃了, 程序 2 退出时, 程序 0 仍然处于阻塞状态, 这说明 `schedule()` 函数被调用时就绪队列是空的。

当孙子进程 (PID 2) 退出时, 它应当唤醒正在等待它的父进程 (PID 1)。虽我们在 `sys_user_exit` 中将父进程的状态改为了 `READY`, 但忘记将父进程重新加入就绪队列 (`insert_to_ready_queue`)。

在 PKE 的调度器设计中, 仅修改 `status` 是不够的, 必须显式调用 `insert_to_ready_queue` 将进程控制块挂入链表, `schedule` 函数才能找到它并调度执行。由于 PID 1 没有被加入队列, PID 1 无法运行, 因此 PID 1 也就无法唤醒 PID 0, 最终导致系统死锁崩溃。

根据 AI 给出的代码, 我们需要修改 `kernel/syscall.c` 中的 `sys_user_exit` 函数, 在唤醒父进程时, 显式调用 `insert_to_ready_queue`, 修正后代码如下：

```

extern void insert_to_ready_queue (process* proc);
ssize_t sys_user_exit (uint64 code) {
    sprint ("User exit with code:%d.\n", code);
    // 1. 将自身设为 ZOMBIE
    current->status = ZOMBIE;
    // 2. 检查父进程是否在等待自己
    if (current->parent && current->parent->status == BLOCKED)
    {
        int64 wait_pid = current->parent->waiting_pid;
        // 如果父进程在等我 (wait_pid == current->pid)
        // 或者父进程在等任意子进程 (wait_pid == -1)
        if (wait_pid == -1 || wait_pid == current->pid) {
            // A. 修改父进程状态为 READY
            current->parent->status = READY;
            // B. 将父进程的 a0 寄存器设置为子进程的 PID (作为 wait 的返回值)
            current->parent->trapframe->regs.a0 = current->pid;
            // [关键修复] C. 将父进程加入就绪队列!
            // 缺少这一步会导致 schedule () 找不到父进程, 从而报 "ready queue empty" 错误
            insert_to_ready_queue (current->parent);
            // D. 将自己设为 FREE (资源回收)
            current->status = FREE;}}
    // 3. 转调度
    schedule ();
    return 0;
}

```

再次编译并运行程序, 得到图 3-7 所示结果, 顺序是孙子先退出, 然后是子进程, 最后是父进程, 且 **flag** 变量值互不影响。与预期结果相符, 成功通过头歌测试, 如图 3-8 所示。

```
root@015b67a0fbff:/app/riscv-pke# spike ./obj/riscv-pke ./obj/app_wait
will fork a child from parent 0.
in alloc_proc. user frame 0x000000087fae000, user stack 0x000000007ffff000, user kstack 0x0000000087fad
going to insert process 1 to ready queue.
going to schedule process 1 to run.
User call fork.
will fork a child from parent 1.
in alloc_proc. user frame 0x000000087fa1000, user stack 0x000000007ffff000, user kstack 0x0000000087fa0
going to insert process 2 to ready queue.
going to schedule process 2 to run.
Grandchild process end, flag = 2.
User exit with code:0.
going to insert process 1 to ready queue.
going to schedule process 1 to run.
Child process end, flag = 1.
User exit with code:0.
going to insert process 0 to ready queue.
going to schedule process 0 to run.
Parent process end, flag = 0.
User exit with code:0.
no more ready processes, system shutdown now.
System is shutting down with exit code 0.
```

图 3-6 lab3_challenge1 运行结果

第1关 lab3_challenge1: (挑战实验) 进程等待和数据段复制

学员代码

代码修改行数 368

kernel/strap.c

复制代码

```
1  /*
2  * Utility functions for trap handling in Supervisor mode.
3  */
4
5  #include "riscv.h"
6  #include "process.h"
7  #include "strap.h"
8  #include "syscall.h"
9  #include "pmm.h"
10 #include "vmm.h"
11 #include "sched.h"
12 #include "util/functions.h"
13 #include "memlayout.h"
14 #include "spike_interface/spike_utils.h"
15
16 //
```

评测次数	详细信息	评测耗时(S)	内存消耗(MB)	评测时间	操作
1	共有2组测试集, 其中有2组测试结果不匹配	5.08	410.29	2025-12-09 19:15	查看代码
2	共有2组测试集, 其中有2组测试结果不匹配	2.27	411.96	2025-12-09 19:17	查看代码
3	评测通过	2.91	411.96	2025-12-09 19:19	查看代码

图 3-7 lab3_challenge1 头歌评测详情

最后我们对 Debug 工作做如下总结, 在之前的调试过程中, 主要遇到了三个关键问题, 导致程序无法编译或运行时崩溃:

1. 编译错误 implicit declaration of function wait:
- 原因: 只在内核实现了 wait 系统调用, 但未在用户态库 (user/user_lib.h 和 .c) 中声明和封装该函数, 导致用户程序无法调用。
- 修复: 在用户库中添加声明和系统调用封装。

2. 编译错误 procs undeclared:

原因: `kernel/syscall.c` 试图访问定义在 `kernel/process.c` 中的全局数组 `procs`, 但缺少外部变量声明。

修复: 在 `kernel/syscall.c` 头部添加 `extern process procs[NPROC];`。

3. 运行时崩溃 `ready queue empty` (孙子进程退出时):

原因: 这是最关键的逻辑错误。当孙子进程退出并试图唤醒父进程时, 代码中仅仅将父进程的状态修改为 `READY`, 但忘记调用 `insert_to_ready_queue` 将其放回就绪队列。导致调度器 `schedule()` 找不到可运行的进程, 从而抛出 `panic`。

修复: 在 `sys_user_exit` 中唤醒父进程时, 显式调用 `insert_to_ready_queue(current->parent)`。

3.3.3 实验建议

1. 建立系统调用注册的自查清单

在 `lab3_2` 调试中发现, 遗漏在 `do_syscall` 中注册分支是常见错误。建议在实验形式上, 要求学生首先提交一份“系统调用链路清单”(涵盖用户态封装、系统调用号定义到内核态分发), 作为一种强制性的设计训练, 减少低级逻辑遗漏。

2. 强化就绪队列状态的一致性检查

在 `challenge1` 中, 仅修改 `status` 而未入队导致死锁。建议内核框架在 `schedule()` 函数中增加断言逻辑: 若发现无就绪进程且有进程处于 `BLOCKED` 态, 应触发警告并显示等待关系, 帮助学生快速定位唤醒逻辑的缺失。

3. 增加数据隔离的显式测试

对于 `fork` 中的数据段深拷贝, 建议设计一个测试用例, 要求父子进程对同一个全局变量进行大量并发修改并校验最终值, 从而让学生深刻理解 `Deep Copy` 在进程隔离中的关键作用。

实验四 文件系统

4.1 实验目的

实验四以文件系统为主题，旨在引导学生理解操作系统中持久化数据管理的基本原理以及文件系统在内核中的整体架构。通过在 PKE 操作系统中实现并使用文件系统相关功能，学生可以从内核实现的角度认识文件、目录、路径解析以及文件系统抽象层（VFS）的设计思想。

与前几章主要关注处理器特权级切换、内存管理和进程调度不同，文件系统实验强调操作系统对外部存储资源的抽象与统一管理能力。实验通过同时支持 `hostfs`（主机文件系统）和 `RFS`（基于内存盘的简易文件系统），使学生理解多文件系统共存场景下，虚拟文件系统如何屏蔽底层差异、向上提供统一接口。

具体目的包括：

1. 掌握 VFS 抽象机制：理解 `vinode`、`dentry`、`super_block` 等核心数据结构如何实现对多种具体文件系统的统一管理。
2. 理解 RFS 磁盘布局：掌握传统块设备文件系统在磁盘（RAM Disk）上的组织方式，包括超级块（Superblock）、索引节点（Inode）、位图（Bitmap）数据块的分配与管理。
3. 实现核心文件操作：通过补全 `RFS` 的文件创建、目录读取及硬链接逻辑，深入理解文件名与 Inode 的映射关系。
4. 探索路径解析与进程环境：通过实现相对路径支持，理解进程当前工作目录（CWD）的维护以及对特殊路径标识符（. 和 ..）的解析逻辑。

4.2 实验内容

本实验围绕 PKE 实验四展开，涵盖了文件系统的基础操作与高级路径解析功能。实验内容主要包括以下三个基础实验和一个挑战实验：

1. `lab4_1` 文件操作：完善 `rfs_create` 函数，实现对新分配磁盘索引节点（`dinode`）的初始化（包括大小、类型、链接数等字段的设置）。
2. `lab4_2` 目录文件：补全 `rfs_readdir` 函数，实现将 `RFS` 目录项（`rfs_dirent`）中的信息复制到 VFS 的通用目录结构中，从而支持目录遍历功能。
3. `lab4_3` 硬链接：实现 `rfs_link` 函数，通过增加目标 Inode 的引用计数并

向父目录添加新目录项，实现多个文件名指向同一物理文件的机制。

4. lab4_challenge1 相对路径：修改路径解析核心函数 `lookup_final_dentry`，支持基于进程 `cwd` 的路径查找，并能正确处理 `.` 和 `..`。

时间信息上，实验三 12 月 10 号到 12 号三天完成，与头歌上记录的时间一致。

实验过程使用 AI 辅助开发，具体实验过程将在 4.3 介绍。

4.3 实验调试及心得

4.3.1 实验收获

1. 理解 VFS 的多态性设计

通过实验，我深刻体会到 VFS 如何通过 `inode_ops` 定义一组函数指针（如 `viop_read`、`viop_write`），并在运行时根据文件所属的文件系统调用具体的实现（如 `rfs_read` 或 `hostfs_read`）。这种设计模式极大地提高了内核的可扩展性。

2. 明确 Inode 与文件名分离的意义

在 RFS 实验中，我认识到文件名仅存在于目录文件的目录项中，而文件的元数据和物理块索引则保存在 Inode 中。这不仅是硬链接实现的基础（多个目录项指向同一个 Inode 号），也清晰展示了文件系统的逻辑层次。

3. 掌握目录缓存（`dir_cache`）的机制

在 Lab 4.2 中，我理解了 RFS 为提高性能，在打开目录时会将所有目录项读入内存中的 `dir_cache`，后续的 `readdir` 操作直接在缓存中进行偏移访问，避免了频繁的磁盘 I/O。

4.3.2 实验过程（AI 交互和调试过程）

按照要求，这里只写挑战实验部分。

Lab4 挑战实验 1 的具体要求为：

通过修改 PKE 文件系统代码，提供解析相对路径的支持。具体来说，用户应用程序在使用任何需要传递路径字符串的函数时，都可以传递相对路径而不会影响程序的功能。

完成用户层 `pwd` 函数（显示进程当前工作目录）、`cd` 函数（切换进程当前工作目录）。

通过如图 4-2 所示的 prompt，agent（仍然使用 Gemini3.0 pro 模型）自动完

成了对五个文件代码的编辑并成功编译和运行，下面我们对具体处理逻辑进行分析和验证。

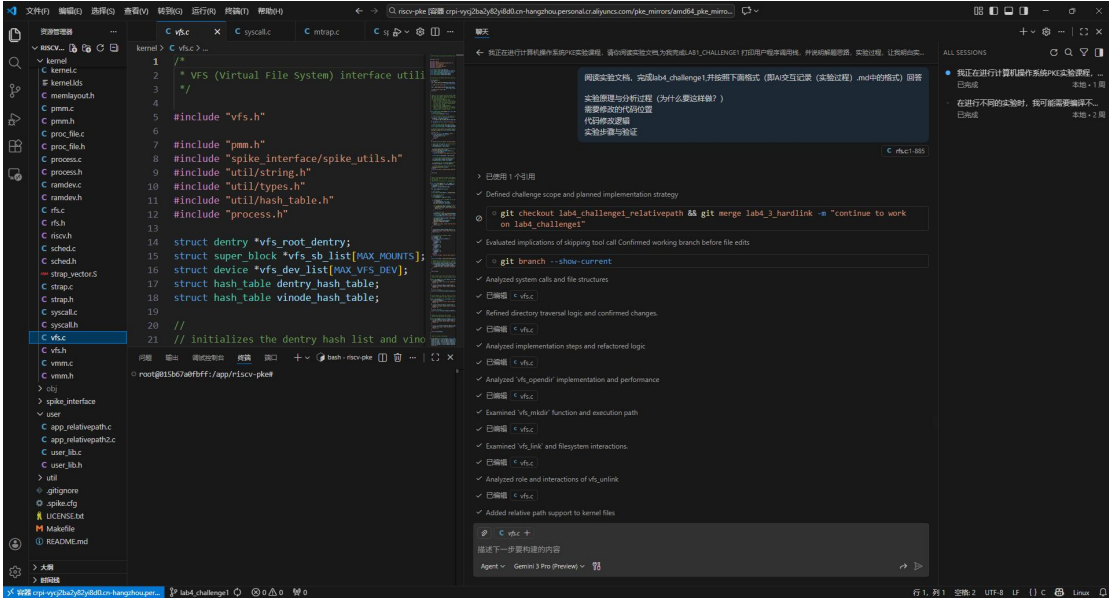


图 4-1 lab4_challenge1 交互记录 1

在文件系统中，路径分为绝对路径（以 / 开头）和相对路径（不以 / 开头）。

绝对路径：总是从根目录（Root Directory）开始解析。

相对路径：从进程的当前工作目录（Current Working Directory, CWD）开始解析。

根据 AI 的分析，实现相对路径支持需要解决三个核心问题：

1. 起点切换：如果路径不以 / 开头，解析的起点应从根目录改为进程的当前工作目录 (CWD)。
2. 特殊标识符处理：在路径解析过程中，遇到 . 应停留在当前 inode，遇到 .. 则需要回溯到当前 inode 的父节点。
3. 环境维护：进程需要能够通过系统调用获取 (rcwd) 和修改 (ccwd) 其当前工作目录

首先，我们需要修改 syscall.h 和 syscall.c，添加系统调用号及分发逻辑：

```
// kernel/syscall.h
#define SYS_user_rcwd (SYS_user_base + 30)
#define SYS_user_ccwd (SYS_user_base + 31)

// kernel/syscall.c
ssize_t sys_user_rcwd(char *pathva){
    char *pathpa = (char*)user_va_to_pa((pagetable_t)(current->p
agetable), (void*)pathva);
```

```

    return do_rcwd(pathpa);
}
ssize_t sys_user_ccwd(char *pathva){
    char *pathpa = (char*)user_va_to_pa((pagetable_t)(current->p
agetable), (void*)pathva);
    return do_ccwd(pathpa);
}

```

// 在 do_syscall switch 中添加 case

然后在 proc_file.c 中实现内核层的 do_rcwd（反向遍历 dentry 树构建路径）和 do_ccwd（打开目录并更新 cwd）：

```

// kernel/proc_file.c
// 读取当前工作目录
int do_rcwd(char *path) {
    struct dentry *d = current->pfiles->cwd;
    char buf[MAX_PATH_LEN];
    int pos = MAX_PATH_LEN - 1;
    buf[pos] = '\0';
    if (d == vfs_root_dentry) {
        strcpy(path, "/");
        return 0;
    }
    // 向上回溯直到根目录
    while (d != vfs_root_dentry && d != NULL) {
        int len = strlen(d->name);
        pos -= len;
        if (pos < 0) return -1;
        memcpy(buf + pos, d->name, len);

        pos--;
        if (pos < 0) return -1;
        buf[pos] = '/';

        d = d->parent;
    }
    strcpy(path, buf + pos);
    return 0;
}
// 切换当前工作目录
int do_ccwd(char *path) {
    struct file *dir_file = vfs_opendir(path);
    if (dir_file == NULL) {
        sprintf("do_ccwd: cannot open directory %s\n", path);

```

```

    return -1;
}

struct dentry *old_cwd = current->pfiles->cwd;
struct dentry *new_cwd = dir_file->f_dentry;

// 更新引用计数并切换 cwd
new_cwd->d_ref++;
current->pfiles->cwd = new_cwd;

// 关闭用于查找的文件句柄
vfs_closedir(dir_file);

// 释放旧 cwd 的引用（通过构造假文件句柄调用 vfs_closedir）
if (old_cwd != vfs_root_dentry) {
    struct file fake;
    fake.f_dentry = old_cwd;
    vfs_closedir(&fake);
}

return 0;
}

```

修改 `vfs.c`: 在 `lookup_final_dentry` 中加入对 `.` 和 `..` 的识别。遇到 `..` 时, 将 `parent` 指向 `parent->parent`。

```

// kernel/vfs.c
struct dentry *lookup_final_dentry(const char *path, struct de
ntry **parent,
                                char *miss_name) {
    // ... (省略部分代码)
    char *token = strtok(path_copy, "/");
    struct dentry *this = *parent;
    while (token != NULL) {
        // 处理 "."
        if (strcmp(token, ".") == 0) {
            token = strtok(NULL, "/");
            continue;
        }
        // 处理 ".."
        if (strcmp(token, "..") == 0) {
            if (this->parent != NULL)

```

```

        this = this->parent;
        token = strtok(NULL, "/");
        continue;
    }

    *parent = this;
    this = hash_get_dentry((*parent), token);
    // ... (后续逻辑不变)

```

调整 VFS 接口起点：修改 `vfs_open` 等函数，判断 `path == '/'`。若为真，起点为 `vfs_root`；否则，起点为 `current->pfile->cwd`。

```

// kernel/vfs.c
// 以 vfs_open 为例，其他函数类似
struct file *vfs_open(const char *path, int flags) {
    struct dentry *parent = vfs_root_dentry;
    // 如果不是绝对路径，则从 cwd 开始
    if (path[0] != '/') {
        parent = current->pfiles->cwd;
    }
    // ...

```

编译并运行测试程序，程序运行后，应该首先显示当前路径为 `/`。随后执行 `cd RAMDISK0`，路径成功变为 `/RAMDISK0`。接着，程序在 `/RAMDISK0` 目录下使用相对路径 `../hostfile.txt` 成功读取了原本位于根目录的文件

```

root@015b67a0fbff:/app/riscv-pke# make clean && make && spike ./obj/riscv-pke ./obj/app_relativepath
Switch to user mode...
in alloc_proc. user frame 0x0000000087f29000, user stack 0x000000007ffff000, user kstack 0x0000000087f28000
FS: created a file management struct for a process.
in alloc_proc. build proc_file_management successfully.
User application is loading.
Application: ./obj/app_relativepath
CODE_SEGMENT added at mapped info offset:3
DATA_SEGMENT added at mapped info offset:4
Application program entry point (virtual address): 0x00000000000100ec
going to insert process 0 to ready queue.
going to schedule process 0 to run.

===== Test 1: change current directory =====
cwd:/
change current directory to ./RAMDISK0
cwd:/RAMDISK0

===== Test 2: write/read file by relative path =====
write: ./ramfile
file descriptor fd: 0
write content:
hello world
read: ./ramfile
read content:
hello world

===== Test 3: Go to parent directory =====
cwd:/RAMDISK0
change current directory to ..
cwd:/
read: ./hostfile.txt
file descriptor fd: 0
read content:
This is an apple.
Apples are good for our health.

All tests passed!

User exit with code:0.
no more ready processes, system shutdown now.
System is shutting down with exit code 0.

```

图 4-2 lab4_challenge1 运行结果

图 4-3 实验结果证明了路径解析器已经具备了动态锚点切换的能力。

通过 .. 的处理，验证了 VFS 目录树的双向遍历功能（即不仅能通过父节点找子节点，也能通过子节点回溯父节点）。

app_relativepath 能够正确打印出切换后的路径，说明 do_rcwd 能够正确遍历内存中的 dentry 树并拼接出字符串。

整个过程无需对底层 RFS 进行大规模修改，体现了 VFS 抽象层的强大灵活性，它通过维护内存中的 dentry 状态，成功为不支持相对路径的底层文件系统提供了高级路径解析支持。

4.3.3 实验建议

1. 标准化进程初始内存段布局

在 lab4_2 调试中，由于 HEAP_SEGMENT 的预分配导致 ELF 加载 offset 不一致，从而无法通过评测。建议在各实验分支中固定 alloc_process 的段初始化顺序，或在评测标准中允许一定范围内的 offset 偏移。

2. 提供 VFS 层级的交互式追踪

文件系统操作涉及 VFS 到具体 FS（如 RFS）的层层调用。建议在 VFS 接口（如 `vfs_open`）中增加可选的追踪日志（Trace Log），显示路径解析每一步生成的 `dentry` 信息，这对于调试相对路径中的 `..` 逻辑非常有帮助。

3. 内容改进建议

`lab4_3` 已经实现了硬链接（基于 `Inode` 号）。可以新增挑战实验：实现软链接。这要求在 `Inode` 中存储目标文件的路径字符串而非 `Inode` 号，并修改 `lookup_final_dentry` 逻辑，使其在解析过程中识别软链接类型并递归地重新开始路径解析。

附录

1. AI prompt 交互过程

由于交互过程太长，实验报告中仅以图片形式展示部分、我将全部交互过程整理成 markdown 格式文档、并存储在源码所在的 gitee 仓库 <https://gitee.com/ilosyi/hustos-pke> 默认分支 lab4_challenge1 的 docs 目录下。

2. 源码

源码存储在远程仓库 <https://gitee.com/ilosyi/hustos-pke>，并含 git 信息，可用于检查代码和 git log 真实性。仓库为私有仓库，并未开源，仅本人和邵老师 gitee 账号可见。